

Computing Information-Gain for Continuous-Valued Attributes

- Let attribute A be a continuous-valued attribute
- Must determine the *best split point* for A
 - Sort the value A in increasing order
 - Typically, the midpoint between each pair of adjacent values is considered as a possible *split point*
 - $(a_i + a_{i+1})/2$ is the midpoint between the values of a_i and a_{i+1}
 - The point with the *minimum expected information requirement* for A is selected as the split-point for A
- Split:
 - D1 is the set of tuples in D satisfying $A \leq \text{split-point}$, and D2 is the set of tuples in D satisfying $A > \text{split-point}$

Gain Ratio for Attribute Selection (C4.5)

- Information gain measure is biased towards attributes with a large number of values.
- It prefers to select attributes having a large number of values.
 - For example, consider an attribute that acts as a unique identifier, such as *product ID*.
 - A split on *product ID* would result in a large number of partitions each one containing just one tuple.
 - The information required to classify data set D based on this partitioning would be $Info_{product\ ID}(D) = 0$.
 - The information gained by partitioning on this attribute is maximal.
 - such a partitioning is useless for classification.

Gain Ratio for Attribute Selection (C4.5)

- C4.5 (a successor of ID3) uses gain ratio to overcome the problem (normalization to information gain)

$$SplitInfo_A(D) = -\sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2\left(\frac{|D_j|}{|D|}\right)$$

- $GainRatio(A) = Gain(A)/SplitInfo(A)$
- Ex.

$$SplitInfo_{income}(D) = -\frac{4}{14} \times \log_2\left(\frac{4}{14}\right) - \frac{6}{14} \times \log_2\left(\frac{6}{14}\right) - \frac{4}{14} \times \log_2\left(\frac{4}{14}\right) = 1.557$$

- $gain_ratio(income) = 0.029/1.557 = 0.019$
- The attribute with the maximum gain ratio is selected as the splitting attribute.

Gini Index

- Gini index measures the impurity of D
- If a data set D contains examples from n classes, gini index, $gini(D)$ is defined as

$$gini(D) = 1 - \sum_{j=1}^n p_j^2$$

- where p_i is the probability that a tuple in D belongs to class C_i and is estimated by $|C_i, D|/|D|$.
 - The sum is computed over n classes.
- The Gini index considers a binary split for each attribute.
- A is a discrete-valued attribute having v distinct values, $\{a_1, a_2, \dots, a_v\}$, occurring in D .
- To determine the best binary split on A , we examine all the possible subsets that can be formed using known values of A .

Gini Index

- Each subset, SA , can be considered as a binary test for attribute A of the form “ $A \in SA$?”
- If A has v possible values, then there are 2^v possible subsets.
 - Income has three possible values, {low, medium, high}
 - The possible subsets are {low, medium, high}, {low, medium}, {low, high}, {medium, high}, {low}, {medium}, {high}, and {}.
 - We exclude the power set, {low, medium, high}, and the empty set from consideration.
 - There are $2^v - 2$ possible ways to form two partitions of the data, D , based on a binary split on A .

Gini Index

- If a data set D is split on A into two subsets D_1 and D_2 , the *gini* index $gini(D)$ is defined as

$$gini_A(D) = \frac{|D_1|}{|D|} gini(D_1) + \frac{|D_2|}{|D|} gini(D_2)$$

- For each attribute, each of the possible binary splits is considered.
- For a discrete-valued attribute, the subset that gives the minimum Gini index for that attribute is selected as its splitting subset.
- For continuous-valued attributes, each possible split-point must be considered.
- The reduction in impurity that would be incurred by a binary split on a discrete- or continuous-valued attribute A is

$$\Delta gini(A) = gini(D) - gini_A(D)$$

Computation of Gini Index

- Ex. D has 9 tuples in buys_computer = “yes” and 5 in “no”

$$gini(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459$$

- Suppose the attribute income partitions D into 10 in D_1 : {low, medium} and 4 in D_2

$$gini_{income \in \{low, medium\}}(D) = \left(\frac{10}{14}\right) Gini(D_1) + \left(\frac{4}{14}\right) Gini(D_2)$$

$$= \frac{10}{14} \left(1 - \left(\frac{7}{10}\right)^2 - \left(\frac{3}{10}\right)^2 \right) + \frac{4}{14} \left(1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2 \right)$$

$$= 0.443$$

$$= Gini_{income \in \{high\}}(D).$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Computation of Gini Index

- The Gini index values for splits on the remaining subsets are
 - 0.458 (for the subsets {low, high} and {medium})
 - 0.450 (for the subsets {medium, high} and {low}).
- The best binary split for attribute income is
 - {low, medium} (or {high}) with Gini index 0.443.
- The best binary split for attribute age is
 - {youth, senior} (or {middle aged}) with Gini index of 0.357.
- For student and credit rating are both binary, with Gini index values of 0.367 and 0.429, respectively.

Computation of Gini Index

- Reduction in impurity

$$\Delta gini(A) = gini(D) - gini_A(D)$$

- $\Delta gini(\text{income})$: $0.459 - 0.443 = 0.016$
- $\Delta gini(\text{age})$: $0.459 - 0.357 = 0.102$
- $\Delta gini(\text{student})$: $0.459 - 0.367 = 0.092$
- $\Delta gini(\text{credit rating})$: $0.459 - 0.429 = 0.030$
- The binary split “age $\in \{\text{youth, senior?}\}$ ” results in the maximum reduction in impurity of the tuples in D
- Age is returned as the splitting criterion.

Comparing Attribute Selection Measures

- The three measures, in general, return good results but
 - **Information gain:**
 - biased towards multivalued attributes
 - **Gain ratio:**
 - tends to prefer unbalanced splits in which one partition is much smaller than the others
 - **Gini index:**
 - biased to multivalued attributes
 - has difficulty when # of classes is large
 - tends to favor tests that result in equal-sized partitions and purity in both partitions

Overfitting and Tree Pruning

- Overfitting: An induced tree may overfit the training data
 - Too many branches, some may reflect anomalies due to noise or outliers
 - Poor accuracy for unseen samples
- Two approaches to avoid overfitting
 - Prepruning:
 - Postpruning:

Tree Pruning

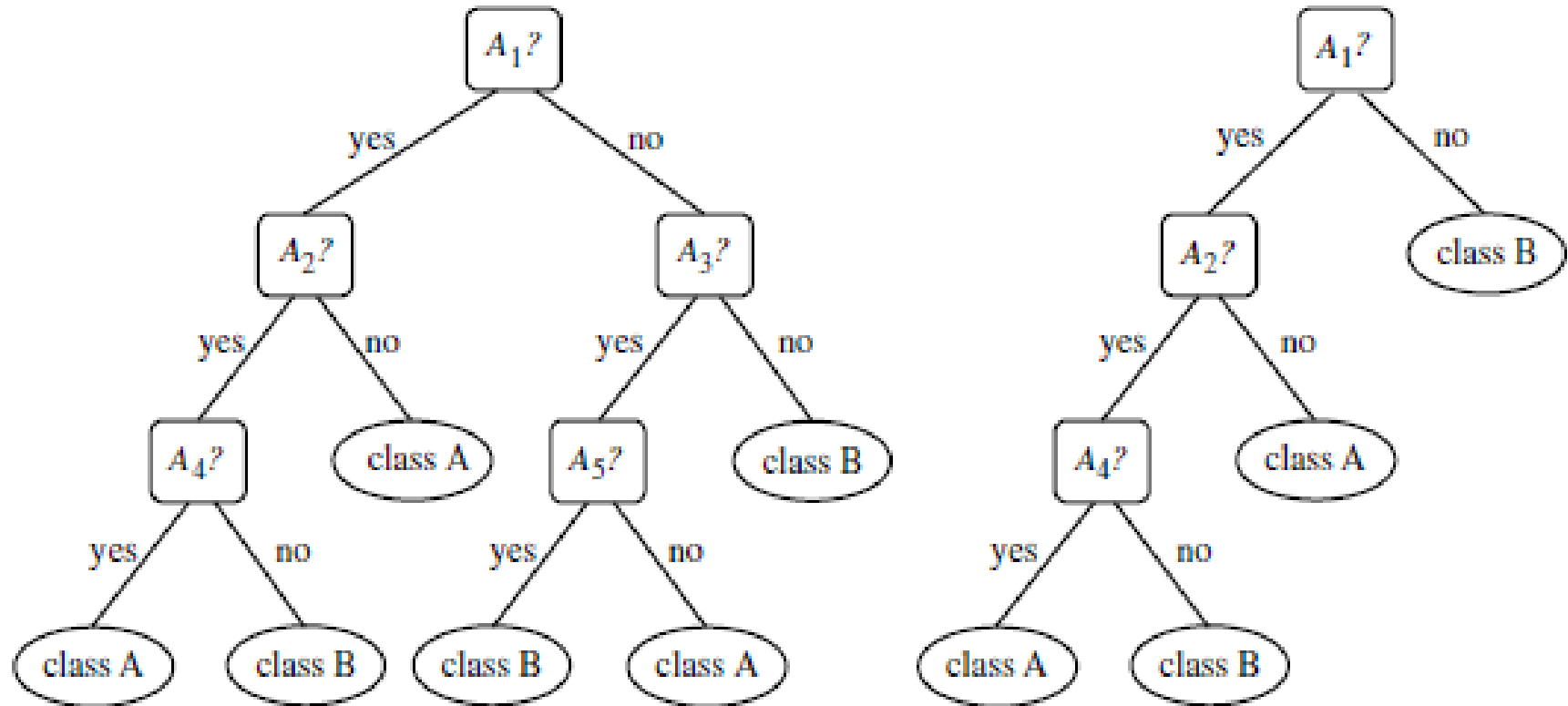
■ Prepruning:

- A tree is “pruned” by halting its construction early.
- Upon halting, the node becomes a leaf.
- The leaf may hold the most frequent class among the subset tuples or the probability distribution of those tuples.
- When constructing a tree, measures such as information gain, Gini index, and so on, can be used to assess the goodness of a split.
- If partitioning the tuples at a node would result in a split that falls below a pre specified threshold, then further partitioning of the given subset is halted.
- There are difficulties, however, in choosing an appropriate threshold.
- High thresholds could result in oversimplified trees, whereas low thresholds could result in very little simplification.

Tree Pruning

- Postpruning:
- which removes subtrees from a “fully grown” tree.
- A subtree at a given node is pruned by removing its branches and replacing it with a leaf.
- The leaf is labeled with the most frequent class among the subtree being replaced.
- Use a set of data different from the training data to decide which is the “best pruned tree”

Tree Pruning



An unpruned decision tree and a pruned version of it.